European Network of
Transmission System Operators
For Electricity

Kubernetes Guide

Energy Communication Platform
Version 4.16.0

Publication Date: 06.05.2025

Energy Communication Platform, version 4.16.0

This software is protected by copyright.

Copyright © 2025, ENTSO-E AISBL
All rights reserved.

This software was developed for ENTSO-E by Unicorn Systems a.s.
www.unicorn.com

Licenses and third-party components:

- This product includes software developed by the Apache Software Foundation (https://www.apache.org/)
- This product includes following third-party open-source components:
  - Apache Tomcat (https://tomcat.apache.org/)
  - Spring Boot (https://spring.io/projects/spring-boot/)
  - ActiveMQ Artemis (https://activemq.apache.org/components/artemis/)
  - Qpid JMS (https://qpid.apache.org/components/jms/)
  - Apache Camel (https://camel.apache.org/)
  - Logback (https://logback.qos.ch/)
  - Spring Framework (https://spring.io/projects/spring-framework/)
  - Hibernate (https://www.hibernate.org/), licensed under LGPL 2.1
  - AngularJS (https://angularjs.org/)
  - Liquibase (https://www.liquibase.org/)

1. Contents

1. Contents ... 3
2. Executive Summary ... 6
3. Revision History ... 7
4. Kubernetes Introduction ... 8
5. Prerequisites ... 9
6. ECP Kubernetes Test Environment ... 10
6.1. Minimum HW Requirements – Test Environment ... 11
6.1.1. Testing Environment Setup ... 12
6.2. Kubernetes Production Environment Requirements ... 12
6.3. Exposed ports ... 13
6.3.1. Endpoint ... 13
6.3.2. Component Directory ... 13
6.3.3. Broker ... 13
6.3.4. ECCo SP Artemis ... 13
7. Helm Charts ... 14
7.1. Structure Description ... 14
7.2. Kubernetes Objects Naming Convention ... 15
7.3. Configuration ... 16
7.3.1. Kubernetes Parameters and HW Sizing ... 16
7.3.2. Application Parameters ... 18
7.3.2.1. Additional Application Parameters - Arrays ... 18
7.3.3. Internal Brokers Configuration: ... 20
7.3.4. Shared Folder Configuration – Custom Volume Mounts ... 20
7.3.5. Global Configuration ... 21
7.3.6. Multiple Instances in a Single Values File ... 21
7.4. Config Maps ... 22
7.4.1. Config Map Endpoint ... 22
7.4.2. Config Map Broker ... 23
7.4.3. Config Map Component Directory ... 24

7.4.4. Config Map Artemis 25
7.4.5. Mapping Custom Config Maps 25
7.5. ECP Cluster Configuration – High Availability Mode 26
7.5.1. Prerequisites for HA Mode 26
7.5.2. ECP Endpoint Cluster Configuration 26
7.5.2.1. ECP Endpoint Shared Folder Configuration 27
7.5.3. ECP Component Directory Cluster Configuration 27
7.5.4. ECP Broker Cluster Configuration 28
7.5.4.1. ECP Broker Shared Folder configuration 29
7.5.5. ECCo SP Artemis Cluster Configuration 30
7.5.5.1. ECCo SP Artemis Shared Folder configuration 30
7.6. ECP Ingress Configuration 31
7.7. ECP Broker Expose AMQPS 33
7.8. Liveness and Readiness Probes 34
7.8.1. Readiness Probe 34
7.8.2. Liveness Probe 34
7.8.2.1. Health Actuator Metrics Configuration 35
8. Deployment to Kubernetes Cluster 36
8.1. Helm Template Rendering 36
8.1.1. Kubernetes Config Kinds 37
8.2. Deployment to Kubernetes Cluster 38
8.3. ECP Components Initialization 38
8.3.1. ECP Component Directory 38
8.3.2. ECP Broker 39
8.3.3. ECP Endpoint 41
8.4. How to Start / Stop Kubernetes Environment 42
8.4.1. ECCo SP Pods Scaling 42
8.4.2. ECCo SP Pod Status Check 43
9. Upgrade 44
9.1. Upgrade Helm Charts 44
9.2. Upgrade Values Files 44

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www. entsoe.eu

9.2.1. Endpoint 44
9.2.1.1. Mandatory Changes 44
9.2.1.2. Optional Changes 45
9.2.2. Component Directory 45
9.2.2.1. Mandatory Changes 45
9.2.2.2. Optional Changes 45
9.2.3. Broker 46
9.2.3.1. Mandatory Changes 46
9.2.3.2. Optional Changes 46
9.2.4. Artemis 46
9.2.4.1. Mandatory Changes 46
9.2.4.2. Optional Changes 47
9.3. Deployment of Upgraded Helm Charts to Kubernetes Cluster 47
10. Useful Kubernetes Commands 48
11. Figure Content 50

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www. entsoe.eu

2. Executive Summary

This document provides Kubernetes Guide for all ECP components.

This document is targeted at ECP Kubernetes Administrators and Users.

3. Revision History

Version	Date	Author	Description
4.11.0	16.05.2023	Drahoslav Stejskal	Document Creation
4.12.0	27.10.2023	Drahoslav Stejskal	Extended by chapters *Upgrade* and *Health Actuator Metrics Configuration*
4.14.0	10.06.2024	Tomáš Kodym	Apache Artemis extension
4.15.0	21.10.2024	Tomáš Kodym	Version updates, minor changes
4.16.0	06.05.2025	Tomáš Kodym	Version updates, minor changes

4. Kubernetes Introduction

Kubernetes is an open-source container orchestration engine for automating deployment, scaling, and management of containerized applications. Kubernetes applications are deployed to unit called *Pod*.

A Pod is a group of one or more containers, with shared storage and network resources, and a specification for how to run the containers. A Pod's contents are always co-located and co-scheduled and run in a shared context. A Pod models an application-specific "logical host": it contains one or more application containers which are relatively tightly coupled.

ECP pod consist of one ECP application and side containers for initialization and Liveness Probes.

For more information about Kubernetes Pods visit official documentation:

- https://kubernetes.io/docs/concepts/workloads/pods/

For easy and automated management of Kubernetes applications, Helm Charts can be used.

Helm Charts helps to define, install, and upgrade even the most complex Kubernetes applications. ECP Helm charts are part of ECP Installation package delivery. Further information about the package manager Helm is available in the official documentation:

- https://helm.sh/
- https://helm.sh/docs/

ECP components can be run within Kubernetes cluster using ECP Docker images from ENTSO-E private Docker repository and some third party docker images available in public repositories. Please, see ECP Installation Guide, chapter *Docker Support* to get further information about Docker for ECP applications.

Using ECP in Kubernetes assumes basic Kubernetes skills for users and Kubernetes administrator for operation of Kubernetes cluster. To get familiar with Kubernetes, please visit official Kubernetes site:

- https://kubernetes.io/
- https://kubernetes.io/docs/setup/
- https://kubernetes.io/docs/tutorials/kubernetes-basics/

Note: While Kubernetes supports more container runtimes than just Docker, Docker is the most commonly known runtime, and it helps to describe Pods using some terminology from Docker.

5. Prerequisites

Kubernetes cluster can be deployed on a local machine or, into the cloud, or for your own datacenter. On client side, Kubectl and Helm needs to be installed.

- Kubernetes for Linux installed on a remote server or in a datacenter. Please see https://kubernetes.io/docs/setup/ for instructions.
  - Kubernetes Production cluster should be installed by experienced Kubernetes Administrator.
- Minikube for installation on a local machine on Linux, Windows or macOS. Please see https://minikube.sigs.k8s.io/docs/start/ for instructions.
  - Minikube is a lightweight Kubernetes implementation that creates a VM on local machine and deploys a simple cluster containing only one node.
  - Minikube can be used for testing purposes.
- The Kubernetes command-line tool kubectl. Kubectl allows to run commands against Kubernetes clusters. Please see https://kubernetes.io/docs/tasks/tools/ for install instructions and https://kubernetes.io/docs/reference/kubectl/kubectl/ for a synopsis of the kubectl.
- Kubernetes deployment and management tool Helm. Please see https://helm.sh/ for instructions. The supported version of Helm is 3.x.
- Access to ECP repositories on Docker Hub (https://hub.docker.com/) – can be granted on request to your existing Docker Hub account.

ENTSO-E AISBL • Rue de Spa 8 • 1000 Brussels • Belgium • Tel + 32 2 741 09 50 • info@entsoe.eu • www. entsoe.eu

6. ECP Kubernetes Test Environment

ECP Kubernetes environment is defined by Values files and Helm charts – see chapter *Helm Charts* for detail information. ECP Helm charts and Values files are available in the ECP installation package:

<ECP Installation package>/ECP Other Deliverables/ ecp-kubernetes-<version>/ecco-sp/

ECP Kubernetes test environment consists of components bellow with pre-configured EIC and instance name:

ECP Component	EIC V-Code	K8s instance name	Values file name
Component Directory	ecp-directory	cd	values-ecp-broker-br.yaml
Broker	ecp-broker	br	values-ecp-directory-cd.yaml
ECP Endpoint	ecp-endpoint-1	ep1	values-ecp-endpoint-ep1.yaml
ECP Endpoint	ecp-endpoint-2	ep2	values-ecp-endpoint-ep2.yaml
ECCo SP Artemis	-	artemis-eptb1	values-eccosp-artemis-eptb1.yaml
ECCo SP Artemis	-	artemis-eptb2	values-eccosp-artemis-eptb2.yaml

Kubernetes environment schema is illustrated on figure below. Please note, that for communication within Kubernetes network, pod's services are used, while for access from outside the Kubernetes cluster, port forwarding to a local machine or Ingress URL with hosts/DNS records must be used.

Note: ECP Kubernetes Test Environment is not initialized. ECP components configuration and registration need to be done manually. In case that own EIC codes/namespace/instance names are used for initialisation, then URLs for services addresses and ECCo SP configuration parameters with reference on EIC codes needs to be changed in Values files.

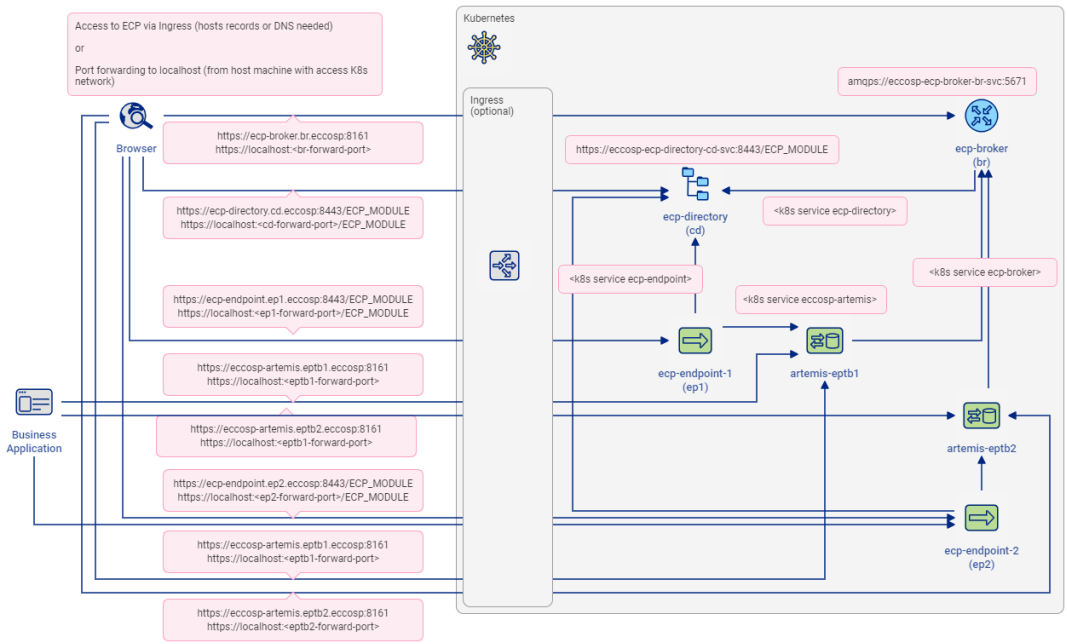

Figure 1: Kubernetes Test Environment

6.1. Minimum HW Requirements – Test Environment

In the table below is stated minimum hardware configuration for Kubernetes ECP test environment hosted on Minikube. The environment consists of 1 Component Directory, 1 Broker and 2 Endpoints. Please note, that the requirements will differ based on requested number of components and expected traffic.

CPU	8 cores – each at least 2Ghz
RAM	16 GB
HDD	*100 GB

** Storage size depends on expected traffic. In case external storage is used, then a majority of disk storage should be allocated to the DB server. Besides storage for Docker containers (binaries from Docker images), ECP Components always need storage for logs and keystore certificates. In case FSSF is used as storage for incoming files, additional disk space needs to be allocated based on the expected data amount and retention configuration.*

6.1.1. Testing Environment Setup

ECP Helm charts were tested on environment with the following setup:

CPU	Master – 2 cores, Worker – 8 cores
RAM	Master – 12 GB, Worker – 48 GB
HDD	Master – 50 GB, Worker – 140 GB
OS	Oracle Linux 8.10
Kubectl client	v1.30.3
Helm	v3.14.0
Docker Engine	V27.5.1
Kubeadm	v1.28.15
Kubernetes (Kubelet)	v1.28.15

6.2. Kubernetes Production Environment Requirements

A production-quality Kubernetes cluster requires planning and preparation. If your Kubernetes cluster is to run critical workloads, it must be configured to be resilient. Configuration of the Kubernetes cluster is out of the scope of this guide. Setup of a Kubernetes production-quality environment is a standard task for experienced Kubernetes Administrator.

The minimum hosts for Kubernetes cluster are:

- Kubernetes Master Node
- Kubernetes Worker Node
  - Best practice is to use at least 3 Nodes
- Operator Node

Some best practice procedures are available online in Kubernetes official documentation:

- https://kubernetes.io/docs/setup/production-environment

6.3. Exposed ports

If the Kubernetes environment contains ECP Component Directory and ECP Broker and other ECP Components are located outside the Kubernetes cluster, then HTTPS interface for sync with Component Directory and AMQPS interface on ECP Broker messaging needs to be exposed.

If the whole ECP network is installed only within Kubernetes network, then access to ECP application GUI can be realized using port forwarding to a local machine.

6.3.1. Endpoint

- 8443 – provides access to web GUI

6.3.2. Component Directory

- 8443 – provides access to web GUI and to the Component Directory API (e.g., for synchronization)

6.3.3. Broker

- 8161 – provides access to web management console of Artemis
- 5671 – AMQPS transport protocol of Artemis

6.3.4. ECCo SP Artemis

- 8161 – provides access to web management console of Artemis
- 5672 – AMQPS transport protocol of Artemis

7. Helm Charts

Helm Charts are Kubernetes YAML manifests combined into a single package that can be easily deployed to Kubernetes cluster. Helm is also providing templates feature so it allows to add variables and use functions inside template files. Helm allows to use various configurations – Values files on a generic template files.

Each ECP component contains its own Helm Charts together with Values files examples. The provided Helm charts can be used for deployment to Kubernetes cluster, after the Values file is configured to define ECP application parameters together with HW sizing specification. It is also possible to use Ingress and service configuration to expose HTTP, HTTPS and AMQPS routes from outside the Kubernetes cluster.

Helm charts provide ability to configure all ECP configuration files, similarly as if the application is installed on VM or server. Values files with configuration are substituted to Helm template, which is used for deployment to the Kubernetes cluster.

ECP Helm Charts are available in the ECP Installation Package in the folder:

- *ECP Other Deliverables/ecp-kubernetes-<version>/ecco-sp*.

7.1. Structure Description

ECP Helm charts are divided by ECP components to separate folders. Each helm chart contains Values file with configuration examples. The only file which is mentioned to be modified by ECP Administrator is Values file.

File Name	Description
templates\configMap.yaml	Holds all ECP configuration files together with default values. Any change in ECP properties files – e.g.: new parameter, new config file, Tomcat config change is reflected there. This file is being updated for each ECP release.
templates\ingress.yaml	Ingress is an object that allows access to Kubernetes services from outside the Kubernetes cluster and configure proxy servers.
templates\service.yaml	Service defines interfaces which are required by pod for communication within Kubernetes cluster - e.g.: amqps, https. Service ports defined within pod configuration.

templates\statefulset.yaml	Definition of statefulset which is used as "recipe" for creation pod instances. It holds information about mounted directories, config files, storages, resources etc.
<component>/Chart.yaml	Holds meta information about helm chart.
Chart.yaml	Holds meta information about each ECCo SP release. Parameter version should be increased for each major configuration change – e.g.: new ECCo SP version. CI tools e.g.: GitLab uses this version for possibility of release rollback.
requirements.yaml	Holds meta information about all ECCo SP helm chart dependencies.
values-<configuration-name>.yaml	Configuration for environment. One values file can hold configuration for all ECCo SP components per environment or each component can be divided to a separate value file. One ECCo SP component can have multiple instances. Any ECCo SP configuration needs to be written to this file. Example of configuration is available in the values files.

7.2. Kubernetes Objects Naming Convention

Kubernetes object names are rendered to template using following naming convention:

- Pod
  - *<Release.Namespace>-<Chart.Name>-<Instance.name>-<Replica.Number>*
    - *e.g.: eccosp-ecp-endpoint-ep1-0*
- StatefulSet
  - *<Release.Namespace>-<Chart.Name>-<Instance.Name>*
    - *e.g.: eccosp-ecp-endpoint-ep1*
- ConfigMap
  - *<Release.Namespace>-<Chart.Name>-<Instance.Name>-cm*
    - e.g.: *eccosp-ecp-endpoint-ep1-cm*
- Service

- *<Release.Namespace>-<Chart.Name>-<Instance.Name>-svc*
  - *e.g.: eccosp-ecp-endpoint-ep1-svc*

- Ingress
  - *<Release.Namespace>-<Chart.Name>-<Instance.name>-ingress*
    - e.g.: *eccosp-ecp-endpoint-ep1-ingress*

- Persistent Volume Claims
  - *<Instance. volumeClaimTemplates .Name> -<Release.Namespace>-<Chart.Name>-<Instance.name>-<Replica.Number>*
    - e.g.: data-*eccosp-ecp-endpoint-ep1-0*

7.3. Configuration

Helm charts configuration defines parameters of ECP components as well as Kubernetes resources. Configuration parameters are being hold in values file in YAML format. To simplify the configuration process, values example files are part of Helm charts delivery.

7.3.1. Kubernetes Parameters and HW Sizing

Kubernetes parameters defines name of instance, Docker image repository URL, resources allocated for the ECP pod and also the size of storages as well as services exposed within Kubernetes cluster. For example:

```
ecp-endpoint: # type of ECP component. Can be ecp-endpoint, ecp-directory, ecp-broker
  enabled: true # boolean flag which defines whether the component will be deployed or not
  instance: # array of ECP instances - one values file can holds multiple instances
    - name: ep1 # name of kubernetes instance
      image: # section for information about Docker image
        name: entsoe/ecp-endpoint # URL to docker image.
        tag: 4.16.0 # Docker image tag - version or 'latest'
      replicaCount: 1 # number of replicas - ECP cluster
      sessionReplication: false # Specifies if the session replication should be activated between ECP EP instances (login, integration purposes)
      resourcesK8s: # section for CPU and memory specification
        requests: # minimum guaranteed amount of a resource that is reserved for a container
          cpu: 500m # minimum CPU - one CPU = 1000m or 1
          ephemeralStorage: 256Mi # minimum ephemeral storage for application
          memory: 1Gi # minimum memory for container - 1Gi is equal to 1024 Mi
        limits: #  maximum amount of a resource to be used by a container.
```

```
Should be always higher than application can allocate (xmx)
          cpu: 1000m # maximum CPU - two CPUs = 2000m or w
          memory: 1512Mi # maximum memory for container
      storageData: 1Gi # embedded database and certificates # storage for
embedded database and certificates. If external DB is used, the value can be
lower
      storageLogs: 256Mi # logs # storage for logs
      keepLogsAfterRestart: true # boolean flag - indicates if ECP logs are
persisted after pod restart, default value false
      livenessProbe: false # automatic restart if pod is not in health state,
default value false
      readinessProbe: true # ECP pod will not started before readiness probe
is fulfilled - check of availability of external DB

      service: # section for definition of services exposed within kubernetes
environment
        https: # protocol used for accessing GUI - can be http or https
          port: 8443 # port used for exposing HTTP(S) service
      ingress: # an object that allows access to Kubernetes services from
outside the Kubernetes cluster.
        enabled: false # boolean flag for enable/disable ingress, default
value false
        contextPath: / # context path for ingress, default value '/'
        host: my.host # URL on which the ECP Endpoint is exposed. Address
needs to be aligned with hosting.
        tls: # section for TLS definition for HTTPS connection to ingress
          secretName: tls-secret  # secret where TLS certificate is stored
        annotations: # ingress annotations section
          kubernetes.io/ingress.class: nginx  # ingress controller definition
```

Please note that memory resources allocated to Kubernetes in section *resourcesK8s* must be always at least 20% higher than resources allocated for (Java) application:

```
ecp-endpoint:
  enabled: true
  instance:
    - name: ep1
…
      resourcesK8s:
        requests: # minimum guaranteed amount of a resource that is reserved
for a container
          cpu: 500m
          ephemeralStorage: 256Mi # minimum ephemeral storage for application
          memory: 1Gi # minimum memory for container - 1Gi is equal to 1024 Mi
        limits: # maximum amount of a resource to be used by a container.
Should be always higher than application can allocate (xmx)
          cpu: 1000m
          memory: 1512Mi # maximum memory for container
...

      envConf: # stands for env.conf file
        resourcesJvm: # memory allocation for ECP component
          xms: 128M
          xmx: 1G # maximum memory for Java application
#         ecpLogFullStackTrace: false # ecp.logFullStackTrace
```

7.3.2. Application Parameters

The Values config contains all configuration files for ECP components. The naming convention for configuration file name and parameters is camelCase instead of “dot case format" where words are divided by dots - used for naming convention inside of the pod.

For example, the ECP Endpoint configuration file ***ecp.properties*** is named as *ecpProperties* in the Values file. For parameters the naming convention is the same. Instead of *spring.profiles.active* in generated standard config file, name *springProfilesActive* is used for Values file.

The mapping of Values parameters to the ECP configuration file is specified within YAML configuration file ***templates/configMap.yaml***. The configuration files are divided by config file name, as shown on the example below – three samples from ***ecp.properties***, ***env.conf*** and ***ecp-users.properties*** are included below.

```
ecp-endpoint: # type of ECP component. Can be ecp-endpoint, ecp-directory,
ecp-broker
  enabled: true # boolean flag which defines whether the component will be
deployed or not

  instance: # array of ECP endpoint instances - one values file can hold
multiple instances
    - name: ep1 # name of kubernetes instance
...
      ecpProperties: # stands for ecp.properties file
        springProfilesActive: ecp-nonha, console-logging #stands for parameter
spring.profiles.active
        dataDirectory: /var/lib/ecp-endpoint # dataDirectory config parameter
for storage of persistent data
        ecpKeystoreLocation: /var/lib/ecp-endpoint/keystore.jks # location of
keystore.jks
...
      envConf: # stands for env.conf file
        resourcesJvm: # memory allocation for ECP component
          xms: 128M
          xmx: 1G
#         ecpLogFullStackTrace: false # ecp.logFullStackTrace

      ecpUsersProperties: # stands for ecp-users.properties
        ecpEndpointUsers: # array of users
          - login: admin # ecp.endpoint.users[0].login
            password: password # ecp.endpoint.users[0].password
            role: admin # ecp.endpoint.users[0].role
          - login: tester # ecp.endpoint.users[1].login
            password: passwordtester # ecp.endpoint.users[1].password
            role: user # ecp.endpoint.users[1].login
…
```

7.3.2.1. Additional Application Parameters - Arrays

Except of standard ECP parameters, some additional parameters are used.
For example, ECP NAT array:

```
ecpNatEnabled: true # NAT is disabled by default - equals to ecp.natEnabled
ecpNatTable: # additional parameter for NAT array. Array values are divided by
```

```
' _ '
  - network: internet # ecp.natTable[0].network
    sourceIp: 1.2.3.4 # ecp.natTable[0].sourceIp
    sourcePort: 8443 # ecp.natTable[0].sourcePort
    destinationIp: 5.6.7.8 # ecp.natTable[0].destinationIp
    destinationPort: 8443 # ecp.natTable[0].destinationPort
  - network: ATOM # ecp.natTable[1].network
    sourceIp: 1.1.1.1 # ecp.natTable[1].sourceIp
    sourcePort: 5671 # ecp.natTable[1].sourcePort
    destinationIp: 2.2.2.2 # ecp.natTable[1].destinationIp
    destinationPort: 5671 # ecp.natTable[1].destinationPort
```

ECP users array:

```
ecpUsersProperties: # stands for ecp-users.properties
  ecpEndpointUsers: # array of users
    - login: admin # ecp.endpoint.users[0].login
      password: password # ecp.endpoint.users[0].password
      role: admin # ecp.endpoint.users[0].role
    - login: tester # ecp.endpoint.users[1].login
      password: passwordtester # ecp.endpoint.users[1].password
      role: user # ecp.endpoint.users[1].login
```

ECCo SP Artemis users array:

```
artemisUsers: # artemis-users.properties
    - login: admin # username for Artemis console
      password: password # password for Artemis console
```

JMX users array:

```
jmxRemotePassword: # jmxremote.password
  jmxRemoteUsers: # array of JMX users
    - login: jmxuser # login for the first JMX user
      password: jmxpassword # password for the first JMX user
      access: readonly # access role for the first JMX user
    - login: jmxadmin # login for second JMX user
      password: jmxadminpassword # password for second JMX user
      access: readwrite # access role for the second JMX user
```

Examples of all available of configuration arrays are included in Values files examples (ecco-sp/values*.yaml).

7.3.3. Internal Brokers Configuration:

In case, that non default internal brokers configuration (JMS) is required for ECP Endpoint component – for example because of increasing memory limit per queue, then the custom internal broker configuration for ECP Endpoint needs to be provided within section volumeMounts within Values file. Please, see ECP Installation guide, chapter *Internal brokers configuration* for further information about the internal brokers configuration.

Configuration of the internal broker – JMS is realised via custom config map – see chapter. *Mapping Custom Config Maps*.

7.3.4. Shared Folder Configuration – Custom Volume Mounts

ECP Helm charts provides possibility for mounting a custom storage (e.g.: shared storage between pods). The Custom volume mounts consists of volume mount definition, where mount path and volume name is defined and volumeClaimTemplates, where the same name must be used and details about storage are defined.

In case that shared folder needs to be established between ECP pods – for example when FSSF interface is used for ECP cluster, mount path to shared folder can be used:

```
ecp-endpoint:
  enabled: true
  instance:
    - name: ep1
...
      volumeMounts: # volume mounts - can be used for mapping config files or
storage
        - name: fssf # name of volume mounts for shared storage
          mountPath: /etc/ecp-endpoint/fssf  # path where to the shared folder
...
```

Volume claim template for shared storage is defined under section volumeClaimTemplates. The name of Volume Claim Template must be the same as defined in section volume mounts. Please note, that the shared storage needs to be of type readWriteMany. Ask you Kubernetes administrator for storage class name for the shared storage.

```
volumeClaimTemplates:
  - metadata:
      name: fssf # volume name
    spec:
      accessModes: [ "ReadWriteMany" ] # storage of type readWriteMany
      storageClassName: nfs-storage # name of storage class - needs to be
provided by Kubernetes administratior
      resources:
        requests:
          storage: 512Mi # request for shared storage size
```

This approach can be used for any custom storage which needs to be mounted to ECP Pod.

7.3.5. Global Configuration

In the bottom of Values file, there is section called *global*, which is common for all ECP instances within the Values configuration.

```
# global config
global
  storage: # storage section
    class: standard # storage class - needs to be provided by Kubernetes
cluster administrator
  imagePullSecrets: # secrets which hold information about login to docker
repository
    - name: regcred # name of secret - name is defined by Kubernees cluster
administrator
```

The global URLs to pull images are defined in default Values file located at path ***<ecp-component>/values.yaml***. If some different repositories (e.g. local) needs to be used instead, these default values can be changed in file ***charts/values-<your values file name>.yaml***.

```
global:
  imageMysqlClient:
    name: arey/mysql-client # URL for MySQL client image pull  - readiness
check for MySQL
  imageMsSqlClient:
    name: mcr.microsoft.com/mssql-tools # URL for MSSQL client image pull -
readiness check for MSSQL
  imagePostgresClient:
    name: bitnami/postgresql:15  # URL for Postgres client image pull -
readiness check for PostgreSQL
  imageOracleClient:
    name: ghcr.io/oracle/oraclelinux8-instantclient:21  # URL for SQLPlus
client image pull - readiness check for Oracle
  imageBusybox:
    name: busybox # URL for busybox image pull - used for ECP side containers
- e.g. file permission fix
```

7.3.6. Multiple Instances in a Single Values File

ECP Helm charts supports multiple instances of ECP components within a single Values file. ECP Instances are divided by instance name:

```
ecp-endpoint:
  enabled: true
  instance:
    - name: ep1
...
  instance:
    - name: ep2
...
```

```
ecp-directory:
  enabled: true
  instance:
    - name: cd
..
ecp-broker:
  enabled: true
  instance:
    - name: br
```

Using this approach allows to define the whole environment within one Values files so it is possible to deploy all components at once. See chapter Deployment to Kubernetes Cluster for further information about deployment.

7.4. Config Maps

Config maps hold all ECP configuration. In the tables below are listed all available config maps and name of sections in Helm files. Mapping of Values sections to ECP config files is defined in file **<component-name>/templates/configMap.yaml** where you can find details about parameters mapping.

The Config Maps from Values file replace ECP configuration files delivered within the Docker image.

7.4.1. Config Map Endpoint

Values Section Name	ECP Configuration File
ecpProperties	ecp.properties
ecpUsersProperties	ecp-users.properties
jmxRemoteProperties	jmxremote.properties
jmxRemotePassword	jmxremote.password
jmxRemotePassword	jmxremote.access
jmxRemoteSsl	jmxremote.ssl

envConf	env.conf setenv.sh
service ecpProperties	server.xml
ecpPasswordProperties	ecp-password.properties
ecpProperties	users.properties
ecpProperties	groups.properties
Custom Config Map	Other configuration files e.g.: ecp-logback.xml

7.4.2. Config Map Broker

Values Section Name	ECP Configuration File
brokerProperties	broker.properties
env	artemis.profile
brokerXml	broker.xml
artemisUsers	artemis-users.properties, artemis-roles.properties
service	bootstrap.xml, jolokia-access.xml
Custom Config Map	Other configuration files e.g.: log4j2.properties

7.4.3. Config Map Component Directory

Values Section Name	ECP Configuration File
ecpDirectoryProperties	ecp-directory.properties
ecpUsersProperties	ecp-users.properties
jmxRemoteProperties	jmxremote.properties
jmxRemotePassword	jmxremote.password
jmxRemotePassword	jmxremote.access
jmxRemoteSsl	jmxremote.ssl
envConf	env.conf setenv.sh
service ecpDirectoryProperties	server.xml
ecpPasswordProperties	ecp-password.properties
ecpProperties	users.properties
ecpProperties	groups.properties
Custom Config Map	Other configuration files e.g.: ecp-logback.xml

7.4.4. Config Map Artemis

Values Section Name	ECP Configuration File
brokerProperties	broker.properties
env	artemis.profile
brokerXml	broker.xml
artemisUsers	artemis-users.properties, artemis-roles.properties
service	bootstrap.xml, jolokia-access.xml
Custom Config Map	Other configuration files e.g.: log4j2.properties

7.4.5. Mapping Custom Config Maps

Besides mapping ECP parameters from Values to Helm template, ECP Kubernetes Helm chars supports also mapping of whole config file. It consists of mapping config map to specified file within the Docker container and content of the desired file inserted to within Values file to the related config map section. Name of mapping volume mounts files is always *config*.

Below is illustrated JMS config file example for embedded DB configuration – **file bootstrap.xml**. Example of the configuration file is included within Values example files for ECP Endpoint (e.g.: values-ecp-endpoint-ep1.yaml)**:**

```
ecp-endpoint:
  enabled: true
  instance:
    - name: ep1
...
      volumeMounts: # volume mounts - can be used for mapping config files or
storage
        - name: config # name of volume mounts for config
          subPath: bootstrap.xml # name of configuration file
          mountPath: /etc/ecp-endpoint/jms/ssl-jks/bootstrap.xml # path where
config file will be mounted
```

The content of the bootstrap file is mapped within Values configuration in section *configMap*. Subpath name for the file must be the same as subPath defined for the file in section mountPath.

Content of the file is inserted to the section content, shifted by two spaces:

```
configMap: # config maps array
  - subPath: bootstrap.xml  # name of config file/volume mount
    content: |- # content is shifted by two spaces
      <?xml version="1.0" encoding="UTF-8"?>
...
```

7.5. ECP Cluster Configuration – High Availability Mode

All ECP components support's cluster configuration. It means that it is possible to run multiple replicas of ECP Pods and therefore to increase availability and reliability of ECP components.

7.5.1. Prerequisites for HA Mode

Prerequisite for installation in the high availability mode is installation of external database which is shared between replicas. Information about requirements on external databases is available in the ECP Installation Guide, chapter *External Databases*. The external database can be installed within Kubernetes cluster (DB Helm charts are not part of delivery package), or on VM/server reachable from the Kubernetes cluster.

7.5.2. ECP Endpoint Cluster Configuration

In the table below are described HA configuration parameters, which are new for Helm charts. The rest of HA parameters follow's camelCase naming format of ECP parameters e.g.: *ecp.db.url* is named as *ecpDBUrl* in Helm chart.

Parameter	Value	Note
replicaCount	2 and more	Number of ECP Endpoint replicas in HA cluster.
ecpDBHostname	DB server address	IP address, hostname, or Kubernetes service name for DB server. This parameter is required for readiness check.

ecpDBName	DB schema name	This parameter is required for readiness check.

Please, see the ECP Installation Guide, chapter *Parameters required for HA installation* to get description of all HA ECP Endpoint Parameters.

Examples of ECP Endpoint configurations for external databases are available in the Kubernetes package, in values file ***values-ecp-endpoint-ep1.yaml***.

7.5.2.1. ECP Endpoint Shared Folder Configuration

In case that the ECP Endpoint cluster uses FSSF integration channel, it is required to establish shared storage for FSSF folders. Add this mount to the section *volumeMounts* in ECP Endpoint Values file:

```
volumeMounts: # volume mounts - can be used for mapping config files or
  - name: fssf # name of volume mounts for shared storage
    mountPath: /etc/ecp-endpoint/fssf  # path where to the shared folder
```

Configure the FSSF folder stated in mount path in as root for FSSF directories in runtime configuration of ECP Endpoint. FSSF Configuration parameters are described in the ECP Administration Guide chapter, *How to configure FSSF*.

For configuration of the shared storage, volume claim template needs to be defined. Volume claim template for shared storage is defined in Values file under section *volumeClaimTemplates*. Ask your Kuberneres cluster administrator to provide you with storage class name with *ReadWriteMany* access mode with sufficient storage space.

```
volumeClaimTemplates:
  - metadata:
      name: fssf # volume name
    spec:
      accessModes: [ "ReadWriteMany" ] # storage of type readWriteMany
      storageClassName: nfs-storage # name of storage class - needs to be
provided by Kubernetes administratior
      resources:
        requests:
          storage: 512Mi # request for shared storage size
```

7.5.3. ECP Component Directory Cluster Configuration

In the table below are described HA configuration parameters, which are new for Helm charts. The rest of HA parameters follow's camelCase naming format of ECP parameters e.g.: *ecp.db.url* is named as *ecpDBUrl* in Helm chart.

Parameter	Value	Note
replicaCount	2 and more	Number of ECP Endpoint replicas in HA cluster.
ecpDBHostname	DB server address	IP address, hostname, or Kubernetes service name for DB server. This parameter is required for readiness check.
ecpDBName	DB schema name	This parameter is required for readiness check.

Make sure that *ecp.urls* in ECP CD runtime configuration is exposed on address of Kubernetes service (common address for all replicas), not on localhost IP address – for example:

ecp.urls = https://eccosp-ecp-directory-cd-svc:8443/ECP_MODULE

ecp.networks = DefaultNetwork

In case that ECP CD needs to be exposed out of the Kubernetes cluster, ECP CD's ingress needs to be configured – see chapter *ECP Ingress Configuration*.

Please, see ECP Installation Guide, chapter *Parameters required for HA installation* to get Description of all HA ECP Component Directory parameters.

Examples of Component Directory configurations for external databases are available in the Kubernetes package, in values file ***values-ecp-directory-cd.yaml***.

7.5.4. ECP Broker Cluster Configuration

In the table below are described HA configuration parameters, which are new for Helm charts. The rest of parameters follow's camelCase naming format of ECP parameters. The chosen strategy for the Artemis HA is Shared Storage. Therefore, the journal storage between the nodes needs to be created.

Parameter	Value	Note
replicaCount	2 and more	Number of ECP Broker replicas in HA cluster.

Make sure that *ecpBrokerUrls* in ECP Broker values file is exposed on address of Kubernetes service (common address for all replicas), not on localhost IP address – for example:

ecpBrokerUrls: amqps://eccosp-ecp-broker-br-svc:5671

ecpBrokerNetworks: DefaultNetwork

In case that ECP Broker needs to be exposed out of the Kubernetes cluster, ECP Broker's Ingress and service needs to be configured – see chapter *ECP Ingress Configuration*.

Please, see the ECP Installation Guide, chapter *Parameters required for HA installation* to get description of all HA ECP Broker parameters.

7.5.4.1. ECP Broker Shared Folder configuration

ECP Broker in cluster needs to use shared storage because of shared authentication, registration certificates and Artemis journal. Shared storage for certificates and configuration has to be enabled by parameter *useSharedStorageForConfiguration*. Size of the storage is configured by parameter *sharedStorageSize* and storage class is defined by parameter *sharedStorageClass*.

Shared storage for journal has to be enabled by parameter: *useSharedStorageForJournal.* Size of the storage is configured by parameter*: sharedStorageSizeJournal* and storage class is defined by parameter: *sharedStorageClassJournal*

Ask your Kuberneres cluster administrator to provide you with storage class name with *ReadWriteMany* access mode with sufficient storage space.

Path to the certificates' shared folder is configured to /opt/ecp-broker/config and for the journal to /opt/ecp-broker/broker/data/.

Configuration example:

```
ecp-broker:
  enabled: true

  instance:
    - name: br
...
      useSharedStorageForConfiguration: true
      sharedStorageSize: 100Mi
      sharedStorageClass: nfs-client
      useSharedStorageForJournal: true
      sharedStorageSizeJournal: 100Mi
      sharedStorageClassJournal: nfs-client
```

7.5.5. ECCo SP Artemis Cluster Configuration

In the table below are described HA configuration parameters, which are new for Helm charts. The rest of parameters follow camelCase naming format of ECP parameters. The chosen strategy for the Artemis HA is Shared Storage. Therefore, the journal storage between the nodes needs to be created.

Parameter	Value	Note
replicaCount	2 and more	Number of ECCo SP Artemis replicas in HA cluster.

In case that ECCo SP Artemis needs to be exposed out of the Kubernetes cluster, ECCo SP Artemis's Ingress and service needs to be configured – see chapter *ECP Ingress Configuration*.

Please, see the ECP Installation Guide, chapter *Parameters required for HA installation* to get description of all HA ECCo SP Artemis parameters.

7.5.5.1. ECCo SP Artemis Shared Folder configuration

ECCo SP Artemis in cluster needs to use shared storage because of shared certificates and Artemis journal. Shared storage for certificates has to be enabled by parameter *useSharedStorageForConfiguration*. Size of the storage is configured by parameter *sharedStorageSize* and storage class is defined by parameter *sharedStorageClass*.

Shared storage for journal has to be enabled by parameter: *useSharedStorageForJournal.* Size of the storage is configured by parameter*: sharedStorageSizeJournal* and storage class is defined by parameter: *sharedStorageClassJournal*

Ask your Kuberneres cluster administrator to provide you with storage class name with *ReadWriteMany* access mode with sufficient storage space.

Path to the shared folder is configured to /opt/ecp-broker/config – the shared certificates must be placed in this folder.

Configuration example:

```
eccosp-artemis:
  enabled: true

  instance:
    - name: artemis-eptb1
...
      useSharedStorageForConfiguration: true
      sharedStorageSize: 100Mi
      sharedStorageClass: nfs-client
      useSharedStorageForJournal: true
      sharedStorageSizeJournal: 100Mi
```

7.6. ECP Ingress Configuration

Kubernetes Ingress is a resource object in Kubernetes that provides an external access point to services running within a Kubernetes cluster. The purpose of Kubernetes Ingress is to route external traffic to the appropriate Kubernetes services based on the URL path or host specified in the incoming request.

In simpler terms, Kubernetes Ingress acts as a reverse proxy and load balancer, allowing external clients to access services running within a Kubernetes cluster without the need for exposing each service individually. Also, the Ingress acts as a single-entry point to the cluster and routes traffic to the correct service based on the rules defined in the Ingress resource.

ECP Helm charts contains support for Ingress, which allows to expose ECP GUI and ECP Broker AMQPS service out of the Kubernetes cluster using domain name. The Domain name records needs to be either imported to DNS server for the environment, or locally to the machine which will access the ECP component on ingress URL.

Usual location for local DNS records:

- */etc/hosts* for Linux
- *C:\Windows\System32\Drivers\etc\hosts* for Windows

The Ingress is not enabled by default, and the ingress section needs to be uncommented and configured in the values file:

```
ingress:
  enabled: true # ingress is activated
   apiVersion: networking.k8s.io/v1 # Api version of ingress
   host: ecp-directory.cd.eccosp # URL on which the ECP is exposed. Address
needs to be aligned with hosting.
   annotations: # ingress annotations section
     kubernetes.io/ingress.class: nginx # ingress annotations - use nginx
     nginx.ingress.kubernetes.io/backend-protocol: "HTTPS" # always use https
- compulsory for remote host tunneling
```

To be able to access the ECP component on URL defined by host, DNS or local hosts file needs to be extended by record containing the host name and Kubernetes cluster IP:

<Kubernetes cluster IP> <ingress hostname>

e.g.: 1.2.3.4 ecp-directory.cd.eccosp

The preconfigured ingresses in the test environment are configured for these addresses:

ECP Component	K8s instance	Ingress URL
Broker	br	https://ecp-broker.br.eccosp:8161 *amqps://ecp-broker.br.eccosp:5671*
Component Directory	cd	*https://ecp-directory.cd.eccosp:8443*
ECP Endpoint	ep1	*https://ecp-endpoint.ep1.eccosp:8443*
ECP Endpoint	ep2	*https://ecp-endpoint.ep2.eccosp:8443*
ECCo SP Artemis	artemis-eptb1	*https:// eccosp-artemis.eptb1.eccosp:8161*
ECCo SP Artemis	artemis-eptb2	*https:// eccosp-artemis.eptb2.eccosp:8161*

Please note, that it is possible to use own ingress address, all you need is to be aligned with DNS hosts records.

To get all ingresses deployed within Kubernetes cluster namespace, run the command below:

kubectl get ingresses -n <namespace>

e.g.: kubectl get ingresses -n eccosp

7.7. ECP Broker Expose AMQPS

In case that ECP Broker's AMQPS interface needs to be exposed out of the Kuberneres cluster, it is needed to use Service of type *LoadBalancer* or *NodePort*. Default Type of Service Type for ECP Broker is *ClusterIP*. Type of service can be changed in Values file in section *service*, parameter *type*. Further information about using Kubernetes services to expose application is available on URL below:

https://kubernetes.io/docs/tutorials/kubernetes-basics/expose/expose-intro/

Configuration example:

```
ecp-broker:
  enabled: true

  instance:
    - name: br
...
      service:
        type: LoadBalancer
        https:
          port: 8161 # Artemis Console GUI
#         nodePort: 30161
        amqps:
          port: 5671 # AMQPS Interface
      ingress:
        enabled: true
         apiVersion: networking.k8s.io/v1
         contextPath: / # context path for ingress, default value '/'
         host: ecp-broker.br.eccosp # URL on which the ECP Broker is exposed.
Address needs to be aligned with hosting.
         annotations: # ingress annotations section
           kubernetes.io/ingress.class: nginx # ingress annotations - use
nginx
           nginx.ingress.kubernetes.io/backend-protocol: "HTTPS" # necessary
for remote host tunelling
           nginx.ingress.kubernetes.io/ssl-passthrough: 'true'
...
      brokerProperties:
...
        ecpBrokerUrls: amqps://eccosp-ecp-broker-br-svc:5671,amqps://ecp-
broker.br.eccosp:5671 # The second ingress URL is for access from external
network
        ecpBrokerNetworks: DefaultNetwork,DefaultNetwork
```

7.8. Liveness and Readiness Probes

Liveness and readiness probes are mechanisms used in Kubernetes to ensure that the containers running in a pod are healthy and ready to serve traffic.

Please, see official documentation if you are interested in detail information about Kubernetes Probes:

- https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

7.8.1. Readiness Probe

All ECP Pods contains optional readiness probe implementation. The implementation is realised via initialization container responsible for checking DB connection to external database, before ECP pod is started (applicable for ECP Endpoint and ECP Component Directory).

This guarantee that ECP Pod will be not started before DB server is initialized. Readiness probe is disabled by default and needs to be enabled by Helm parameter *readinessProbe*:

```
ecp-endpoint:
  enabled: true
  instance:
    - name: ep1
...
      readinessProbe: true # ECP pod will not started before readiness probe
is fulfilled - check of availability of external DB
```

7.8.2. Liveness Probe

A liveness probe is a diagnostic test that determines whether a container is running properly or not. Kubernetes periodically sends an HTTP request to the container, and if it doesn't respond with a successful status code, it will mark the container as failed and restart it. Liveness probes are useful in scenarios where a container may enter a state where it is running, but not able to serve requests.

Implementation for ECP Pods contains check of health status. Health status for ECP Endpoint and Directory is done via health actuator – you can find further information in ECP Administration Guide, chapter *Health Endpoint.* Status for ECP Broker is done via checking listening on port for Artemis web console (default port is 8161).

Health check is executed every 60 seconds with initial delay 120 seconds. Liveness probe is disabled by default and needs to be enabled by Helm parameter *LivenessProbe*:

```
ecp-endpoint:
  enabled: true
  instance:
    - name: ep1
...
      livenessProbe: true # automatic restart if pod is not in health state,
default value false
```

7.8.2.1. Health Actuator Metrics Configuration

ECP Endpoint and ECP Component Directory supports ECP Health Actuator metrics configuration, which is used as input for Liveness Probe in Kubernetes Helm Charts. By default, all metrics are excluded in Helm Config Map parameter ecpHealthExclude.

In the extract below is illustrated metrics exclude in the ECP Endpoint Config Map:

```
ecp.healthExclude={{- $instance.ecpProperties.ecpHealthExclude | default
"connectedBrokers,waitingToDeliver,waitingToReceive,isReloading,endpointStatus
,certificatesStatus,invalidMessagePath,withoutValidMessagePath,
internalBrokerConnected" }}
```

In the following Values configuration example, ECP Endpoint instance *ep1* is configured for Health Actuator based on connection to internal broker (*internalBrokerConnected*).If the connection is not ready, Health Actuator will return HTTP error code 503 and ECP Endpoint pod will be automatically restarted. The metrics *internalBrokerConnected* is not present in the value of *ecpHealthExclude* list, therefore it is included.

```
ecp-endpoint:
  enabled: true
  instance:
    - name: ep1
…
      livenessProbe: true
…
        ecpHealthExclude: connectedBrokers,waitingToDeliver,waitingToReceive
,isReloading,endpointStatus,certificatesStatus,invalidMessagePath,withoutValid
MessagePath
```

For further information about Health Actuator Configuration, please see ECP Administration Guide, chapter *Health Endpoint*.

8. Deployment to Kubernetes Cluster

Deployment to Kubernetes cluster is done via Helm template and Kubectl console. The Helm template is rendered based on Values files, which are substituted to Helm charts.

8.1. Helm Template Rendering

The Helm template holds the complete Kubernetes and application configuration required for deployment to Kubernetes cluster. For template rendering, Helm 3.x is required to be installed locally on the machine, from which rendering will be done. The command *'helm template'* render chart templates locally and display the output.

Syntax for helm template:

helm template [NAME] [CHART] [flags]

For the example below, we will generate helm for ECP Endpoint with these parameters:

- deployment name: *ecp-ep*
- chart name: ecco-sp
- namespace *eccosp*
- values file located at path *ecco-sp\values-ecp-endpoint-ep1.yaml.*
- template will be rendered to file ecco-sp\template-ecp-endpoint-ep1.yaml

Template Example Endpoint:

helm template ecp-ep ecco-sp -f .\ecco-sp\values-ecp-endpoint-ep1.yaml -n eccosp > .\ecco-sp\template-ecp-endpoint-ep1.yaml

Template Example Component Directory:

helm template ecp-cd ecco-sp -f .\ecco-sp\values-ecp-directory-cd.yaml -n eccosp > .\ecco-sp\template-ecp-directory-cd.yaml

Template Example Broker:

helm template ecp-br ecco-sp -f .\ecco-sp\values-ecp-broker-br.yaml -n eccosp > .\ecco-sp\template-ecp-broker-br.yaml

Template Example Artemis:

helm template eccosp-artemis ecco-sp -f .\ecco-sp\values-eccosp-artemis-eptb1.yaml -n eccosp > .\ecco-sp\template-eccosp-artemis-eptb1.yaml

8.1.1. Kubernetes Config Kinds

The rendered template file contains all kinds of Kubernetes configurations which consists of:

- ConfigMap
  - Application configuration.
- Service
  - Service configuration – name, exposed ports, protocols
- StatefulSet
  - Workload API object used to manage stateful applications. Manages the deployment and scaling of a set of Pods and provides guarantees about the ordering and uniqueness of these Pods. Holds information about containers, storages, permissions, labels etc.
- Ingress
  - Ingress configuration that allows access to Kubernetes services from outside the Kubernetes cluster and configure proxy servers.

The rendered template holds information about all components which are included in the Values file. Single Values file can be used for installation of the whole ECP environment or Values file can be divided to smaller sections for example based on component type to make sure that deployment of one ECP component will not affect other ECP pods.

8.2. Deployment to Kubernetes Cluster

There are more options how to deploy application to the Kubernetes cluster. Within this Guide, we will use rendered Helm template as described in previous chapter together with *'apply'* command.

Deployment Example Endpoint:

kubectl apply -f .\ecco-sp\template-ecp-endpoint-ep1.yaml -n eccosp

Deployment Example Component Directory:

kubectl apply -f .\ecco-sp\template-ecp-endpoint-cd.yaml -n eccosp

Deployment Example Broker:

kubectl apply -f .\ecco-sp\template-ecp-endpoint-br.yaml -n eccosp

Deployment Example Artemis:

kubectl apply -f .\ecco-sp\template-eccosp-artemis-eptb1.yaml -n eccosp

Please note, that for application of Helm template configuration file for Kubernetes cluster needs to be installed to kubectl tool (.kube config file). Ask your Kubernetes Administrator for the Kubernetes configuration file.

8.3. ECP Components Initialization

After deployment of ECP components to Kubernetes cluster, it is needed to initialise ECP in the same way as it is described in ECP Installation Guide, extended by Kubernetes steps.

In case that own EIC codes/namespace/instance names are used for initialization, then URLs for services addresses and ECP configuration parameters with reference on EIC codes needs to be changed in Values files.

8.3.1. ECP Component Directory

1. Access ECP Component Directory GUI
   - Tunnel to service port can be created for this purpose
     - *kubectl port-forward eccosp-ecp-directory-cd-0 8443:8443 -n eccosp*
   - The other option is to access the GUI via Ingress.

2. Login to the ECP Component Directory on URL https://localhost:8443/ECP_MODULE
3. Follow the ECP Installation Guide, chapter *Registration process*
   - For ECP CD URL use address of kubernetes service
     - *https://<namespace>-ecp-directory-<instance-name>-svc:<port>/ECP_MODULE*
     - *e.g.: https://eccosp-ecp-directory-cd-svc:8443/ECP_MODULE*
   - The other option is to use ingress address if access from external network is configured
4. Restart of ECP Component Directory Pod is required, before exporting registration certificate for other ECP components.
   - kubectl scale statefulset <statefulset> --replicas=0 -n service -n <namespace>
   - kubectl scale statefulset <statefulset> --replicas=1 -n service -n <namespace>

for example:

- *kubectl scale statefulset eccosp-ecp-directory-cd --replicas=0 -n eccosp*
- *kubectl scale statefulset eccosp-ecp-directory-cd --replicas=1 -n eccosp*

8.3.2. ECP Broker

1. Download registration keystore from ECP Component Directory.
2. Upload the registration keystore to ECP Broker's Pod to file /opt/ecp-broker/config/keystore.jks
   - *kubectl cp <src-path-to-file> <dest-pod>:<dest-path-to-file> -n <namespace>*
   - *e.g.: kubectl cp ./authKeystore.jks eccosp-ecp-broker-br-0:/opt/ecp-broker/config/keystore.jks -n eccosp*
3. Get into the ECP Broker pod
   - *kubectl exec -it -n <namespace> <pod name> -- /bin/bash*
   - *e.g.: kubectl exec -it -n eccosp eccosp-ecp-broker-br-0 -- /bin/bash*
4. Copy ECP Broker configuration file **broker.properties** to a temporary file e.g.: **broker-temp.properties**
   - *cp /opt/ecp-broker/config/broker.properties /opt/ecp-broker/config/broker-temp.properties*
5. Send registration request to ECP CD using the **broker-temp.properties**

- *java -D"spring.config.location=/opt/ecp-broker/config/broker-temp.properties" -jar /opt/ecp-broker/reg/broker-registration-tool.jar -m INITIAL_REGISTRATION*

6. Approve the registration request on the ECP Component Directory GUI
7. Check the status of the registration request by running the command:
   - *java -D"spring.config.location=/opt/ecp-broker/config/broker-temp.properties" -jar /opt/ecp-broker/reg/broker-registration-tool.jar -m PROCESS_REGISTRATION*
   - To continue with next step the output from console should be:
     - *"Request with ID {requestID} is approved. Storing signed AUTHENTICATION*
8. PUSH ECP Broker configuration – the *requestID* was stored as value of parameter ecp.broker.registrationDirectoryId
   - *java -D"spring.config.location=/opt/ecp-broker/config/broker-temp.properties" -jar /opt/ecp-broker/reg/broker-registration-tool.jar -m PUSH_CONFIGURATION*
9. Fill-in value of parameter ecp.broker.registrationDirectoryId from broker-temp.properties file to the Values file for ECP Broker (example file **values-ecp-broker-br.yaml**) – parameter *ecpBrokerregistrationDirectoryId* in section brokerProperties
10. Render new version of Broker template and apply it to Kubernetes cluster. See chapter *Deployment to Kubernetes Cluster*.
11. Restart ECP Broker pod – scale pod to 0, wait until pod is terminated, scale the pod to value 1

- *kubectl scale statefulset <statefulset> --replicas=0 -n service -n <namespace>*
- *kubectl scale statefulset <statefulset> --replicas=1 -n service -n <namespace>*

for example:

- *kubectl scale statefulset eccosp-ecp-broker-br --replicas=0 -n eccosp*
- *kubectl scale statefulset eccosp-ecp-broker-br --replicas=1 -n eccosp*

12. Connect to the ECP Broker - create a tunnel to Artemis console – default port is 8161
    - *kubectl port-forward eccosp-ecp-broker-br-0 8161:8161 -n eccosp*
    - *Login to Artemis console URL* https://localhost:8161

- Use credentials defined in ECP Broker Values file (example file ***values-ecp-broker-br.yaml***) – in section *artemisUsers*. If no credentials are filled in default login, the one defined in file */opt/ecp-broker/broker/etc/artemis-users.properties* is used.

> The other option is to access the console via Ingress.

13. Check ECP Broker queues

> *Queues* tab

- *There will be visible queues for registered ECP Endpoints with configured ECP Message path via the ECP Broker*

8.3.3. ECP Endpoint

1. Download registration keystore from ECP Component Directory
2. Access ECP Endpoint GUI

> Tunnel to service port can be created for this purpose

- *kubectl port-forward <pod-name> <local port>:<pod port> -n <namespace>*
- *e.g.: kubectl port-forward eccosp-ecp-endpoint-ep1-0 8443:8443 -n eccosp*

> The other option is to access the GUI via Ingress

3. Follow ECP Installation Guide, chapter *Registration process*

> For ECP CD URL use address of kubernetes service

- *https://<namespace>-ecp-directory-<instance-name>-svc:<port>/ECP_MODULE*
- *e.g.: https://eccosp-ecp-directory-cd-svc:8443/ECP_MODULE*

> The other option is to use Ingress address if access from external network is configured

4. Configure message path – follow the *ECP Installation Guide*, chapter *Configure message path*
5. Execute Messaging Connectivity test from the ECP Endpoint GUI

- To test the messaging connectivity, go to the settings page, section Messaging connectivity and click the *"Check Connectivity"* button. This button will redirect you to the Connectivity Check page. Choose Receiver's ECP Endpoint code from the dropdown menu, fill the message type you want to test and click the Check button. When connectivity check is done, the status will be displayed right above the Check button.
    - To get further information about connectivity test, please refer to the ECP Functional Specification, chapter *UC Connectivity Test*.

8.4. How to Start / Stop Kubernetes Environment

For start/stop of ECP components in K8s, kubectl console can be used. The other approach would be to use some IDE – e.g.: OpenLens or orchestration platform e.g.: Rancher.

8.4.1. ECCo SP Pods Scaling

ECP Kubernetes Pods are Controlled by StatefulSet. For ECP Instance stopping, count of replicas needs to be scaled to 0. For ECP Instance start-up, count of replicas needs to be changed to 1 or higher value (if multiple replicas in cluster are configured).

Operation	Command	Example
Scale statefulset – turn pod off	*kubectl scale statefulset <statefulset > --replicas=0 -n service -n <namespace>*	*kubectl scale statefulset eccosp-ecp-endpoint-ep1 –replicas=0 -n service -n eccosp*
Scale statefulset – turn pod on	*kubectl scale statefulset <statefulset > --replicas=1 -n service -n <namespace>*	*kubectl scale statefulset eccosp-ecp-endpoint-ep1 –replicas=1 -n service -n eccosp*

8.4.2. ECCo SP Pod Status Check

Status of ECP pod can be validated via kubectl command *'get pods'*:

Operation	Command	Example
Get all pods from namespace	*kubectl get pods -n <namespace>*	*kubectl get pods -n eccosp*

- If the ECP Pod is stopped, it is not visible in the list of Pods in namespace.
- If the ECP Pod is running, it is returned in the list of the Pods with status *RUNNING*.

The other approach is to check status of statefulset via kubectl command *'get statefulsets'*:

Operation	Command	Example
Get all statefulsets	*kubectl get statefulsets -n <namespace>*	*kubectl get statefulsets -n eccosp*

- If the ECP Pod is stopped, then there is value '0/0' for column '*READY*' on the same line with ECP component's statefulset.
- If the ECP Pod is stopped, then there is value '1/1' or higher based on configured replicas count for column '*READY*' on the same line with ECP component's statefulset.

9. Upgrade

Kubernetes upgrade consists of upgrade of helm charts, update of values files, rendering a new template and deployment to Kubernetes cluster.

9.1. Upgrade Helm Charts

All helm files need to be replaced for each release (helm files are listed in chapter *Structure Description*).

Replace the whole Helm charts resources except Values file from previous release with the new Helm charts delivery.

ECP Helm Charts are available in the ECP Installation Package in the folder:

> *ECP Other Deliverables/ecp-kubernetes-<version>/ecco-sp*.

Values Files Update is described in chapters below.

9.2. Upgrade Values Files

For upgrading existing ECP Helm Charts at least docker image tag (version) needs to be updated to the current release version.

Existing values files for components needs to be updated if new mandatory parameters are added or if parameters from newly added functionality needs to be configured.

9.2.1. Endpoint

Changes below needs to be done in all values files for ECP Endpoint component. Please make sure to apply the changes to all instances if multiple instances are located within the single Values file.

9.2.1.1. Mandatory Changes

> Locate helm parameter(s) ‘tag’ in section image for component ecp-endpoint, change value of tag to 4.16.0.

9.2.1.2. Optional Changes

Optional changes for release are available in section *'Spring Release 2025'* the values file example:

- values-ecp-endpoint-ep1.yaml

Summary of optional changes:

- Added support for GUI session replication, when deployed after loadbalancer.
- Added possibility to logging to console (stdout).
- Added new parameter for configuration of message priority.
- Added new parameters for LDAP configuration.
- Added new parameters for configuration of connection to broker.
- Added possibility to enable/disable authentication for actuator endpoints.

Please, see the chapter *New configuration properties* chapter in the ECP Upgrade Guide with a table of new configuration properties added since the last ECP release.

Please note, that camelCase naming format of ECP parameters is used for Values file parameters (e.g.: *ecp.db.url* is named as *ecpDBUrl*).

9.2.2. Component Directory

Changes below needs to be done in all Values files for ECP Directory component. Please make sure to apply the changes to all instances if multiple instances are located within the single Values file.

9.2.2.1. Mandatory Changes

- Locate helm parameter(s) 'tag' in section image for component ecp-directory, change value of tag to 4.16.0.

9.2.2.2. Optional Changes

Optional changes for release are available in section *'Spring Release 2025'* the values file example:

- values-ecp-directory-cd.yaml

Summary of optional changes:

- Added new parameters for LDAP configuration.
- Added possibility to logging to console (stdout).

> Added possibility to enable/disable authentication for actuator endpoints.

Please see the chapter *New configuration properties* chapter in the ECP Upgrade Guide with a table of new configuration properties added since the last ECP release.

Please note that camelCase naming format of ECP parameters is used for Values file parameters (e.g.: *ecp.db.url* is named as *ecpDBUrl*).

9.2.3. Broker

Changes below needs to be done in all values files for ECP Broker component. Please make sure to apply the changes to all instances if multiple instances are located within the single Values file. The messaging engine has been moved from the ActiveMQ to the Artemis. Therefore, before any upgrade, make sure that all messages are processed, as the Artemis does not use external DB anymore, but shared storage only.

9.2.3.1. Mandatory Changes

> Locate helm parameter(s) 'tag' in section image for component ecp-broker, change value of tag to 4.16.0.

9.2.3.2. Optional Changes

All parameters including optional changes for release are available in the values file example:

> *values-ecp-broker-br.yaml*

Summary of optional changes:

> No changes.

9.2.4. Artemis

Changes below needs to be done in all Values files for ECCo SP Artemis component. Please make sure to apply the changes to all instances if multiple instances are located within the single Values file.

9.2.4.1. Mandatory Changes

> Locate helm parameter(s) 'tag' in section image for component eccosp-artemis, change value of tag to 4.16.0.

9.2.4.2. Optional Changes

All parameters including optional changes for release are available in the values file example:

> *values-eccosp-artemis-eptb1.yaml*

Summary of optional changes:

> No changes.

9.3. Deployment of Upgraded Helm Charts to Kubernetes Cluster

When Helm Chart Resources and Values files for all components are updated, the new Helm Charts needs to be deployed to Kubernetes Cluster. Firstly, generate new Helm Templates and then Deploy the upgraded Templates to Kubernetes Cluster.

Please, follow chapters *Helm Template Rendering* and *Deployment to Kubernetes Cluster*.

After the deployment of new Helm Charts version, ECP pods should be automatically restarted in order to apply changes. If ECP pods are not restarted, restart them manually – for example by scaling count of instances to 0 and then back to original count of instances – as described in chapter *ECCo SP Pods Scaling.*

10. Useful Kubernetes Commands

In the table below are listed some basic commands for management of Kubernetes cluster. See official documentation if you are interested in more Kubernetes commands. In the commands below, namespace “eccosp” and instance name “ep1” was used.

- https://kubernetes.io/docs/reference/kubectl/cheatsheet/
- https://kubernetes.io/docs/reference/kubectl/overview/

Operation	Command	Example
Get into a pod	*kubectl exec -it -n <namespace> <pod name> -- /bin/bash*	*kubectl exec -it -n eccosp eccosp-ecp-endpoint-ep1-0 -- /bin/bash*
Forward a local port to a port on the Pod	*kubectl port-forward <pod-name> <local port>:<pod port> -n <namespace>*	*kubectl port-forward eccosp-ecp-endpoint-ep1-0 84431:8443 -n eccosp*
Scale statefulset - turn pod off	*kubectl scale statefulset <statefulset > --replicas=0 -n service -n <namespace>*	*kubectl scale statefulset eccosp-ecp-endpoint-ep1 --replicas=0 -n service -n eccosp*
Scale statefulset - turn pod on	*kubectl scale statefulset <statefulset > --replicas=1 -n service -n <namespace>*	*kubectl scale statefulset eccosp-ecp-endpoint-ep1- --replicas=1 -n service -n eccosp*
Get logs for pod	*kubectl log <pod-name> -n <namespace>*	*kubectl log eccosp-ecp-endpoint-ep1-0 -n eccosp*
Copy data from pod	*kubectl cp <src-pod>:<src-path-to-file> <dest-path-to-file> -n <namespace>*	*kubectl cp eccosp-ecp-endpoint-ep1-0:/tmp/ecp.properties ./ecp.properties -n eccosp*
Copy data to pod	*kubectl cp <src-path-to-file> <dest-pod>:<dest-path-to-file> -n <namespace>*	*kubectl cp ./ecp.properties eccosp-ecp-endpoint-ep1-0:/tmp/ecp.properties -n eccosp*
Get config maps	*kubectl get configmaps -n <namespace>*	*kubectl get configmaps -n eccosp*
Edit configmap runtime	*kubectl edit cm -n <namespace> <config-map-name>*	kubectl edit cm -n eccosp eccosp-ecp-endpoint-ep1-cm

Get all available contexts	*kubectl config get-contexts*	-
Set context	*kubectl config use-context <name>*	*kubectl config use-context eccosp*
Get current context	*kubectl config current-context*	-
Get namespace	*Kubectl get namespace*	-
Get all pods from namespace	*kubectl get pods -n <namespace>*	*kubectl get pods -n eccosp*
Get all statefulsets	*kubectl get statefulsets -n <namespace>*	*kubectl get statefulsets -n eccosp*
Get all physical volume claims	*kubectl get pvc -n <namespace>*	*kubectl get pvc -n eccosp*

11. Figure Content

Figure 1: Kubernetes Test Environment 11